

MODULE II • LECTURE 04

Backpropagation & Optimization

How Neural Networks Learn Their Weights

Dr Tamal Ghosh

Recap: What Have We Built?

The Forward Pass

A feedforward neural network performs:

$$x \rightarrow Wx + b \rightarrow f(\cdot)$$

Multiple layers allow complex nonlinear transformations.

- ▶ Activation functions (ReLU, Sigmoid, Softmax)
- ▶ Dropout & Network Capacity
- ▶ Overfitting mitigation

The Missing Piece

We have designed the architecture.

But one crucial question remains:

How does the network learn

W and b ?

Learning as an Optimization Problem

$$\theta^* = \arg \min_{\theta} L(\theta)$$

The Core Objective

Finding the Best Parameters

A neural network is defined by its parameters:

$$\theta = \{ W, b \}$$

We define a loss function $L(\theta)$

Training attempts to find the specific set of parameters that **minimizes this loss**.

This is fundamentally an optimization problem.

What Is Loss?

Loss measures how far the prediction is from the true target.



Binary Cross-Entropy

For binary classification, a common choice is:

$$L = - [y \log (\hat{y}) + (1 - y) \log (1 - \hat{y})]$$



Variables

- ▶ y = true label (0 or 1)
- ▶ $\hat{y}$ = predicted probability

Lower loss → better prediction.

A Simple Learning Example

Scenario A: Good Prediction

Suppose the true target is:

$$y = 1$$

And the model predicts:

$$\hat{y} = 0.9$$

The loss is very small.

Scenario B: Bad Prediction

If the true target is still $y = 1$

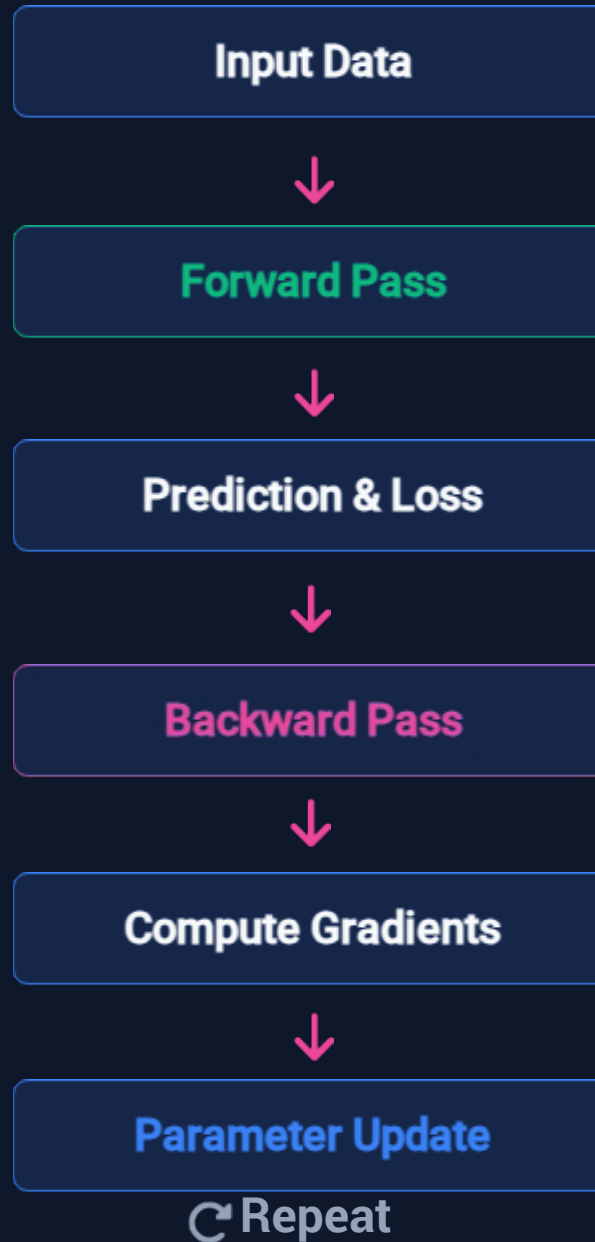
But the model predicts:

$$\hat{y} = 0.1$$

The loss is much larger.

Training tries to modify the parameters so that **future predictions reduce the loss.**

This loop is the foundation of neural-network training.





Forward Propagation

Computing the prediction from the input.

$$x \rightarrow z_1 \rightarrow a_1 \rightarrow z_2 \rightarrow \hat{y}$$

Consider a simple network:

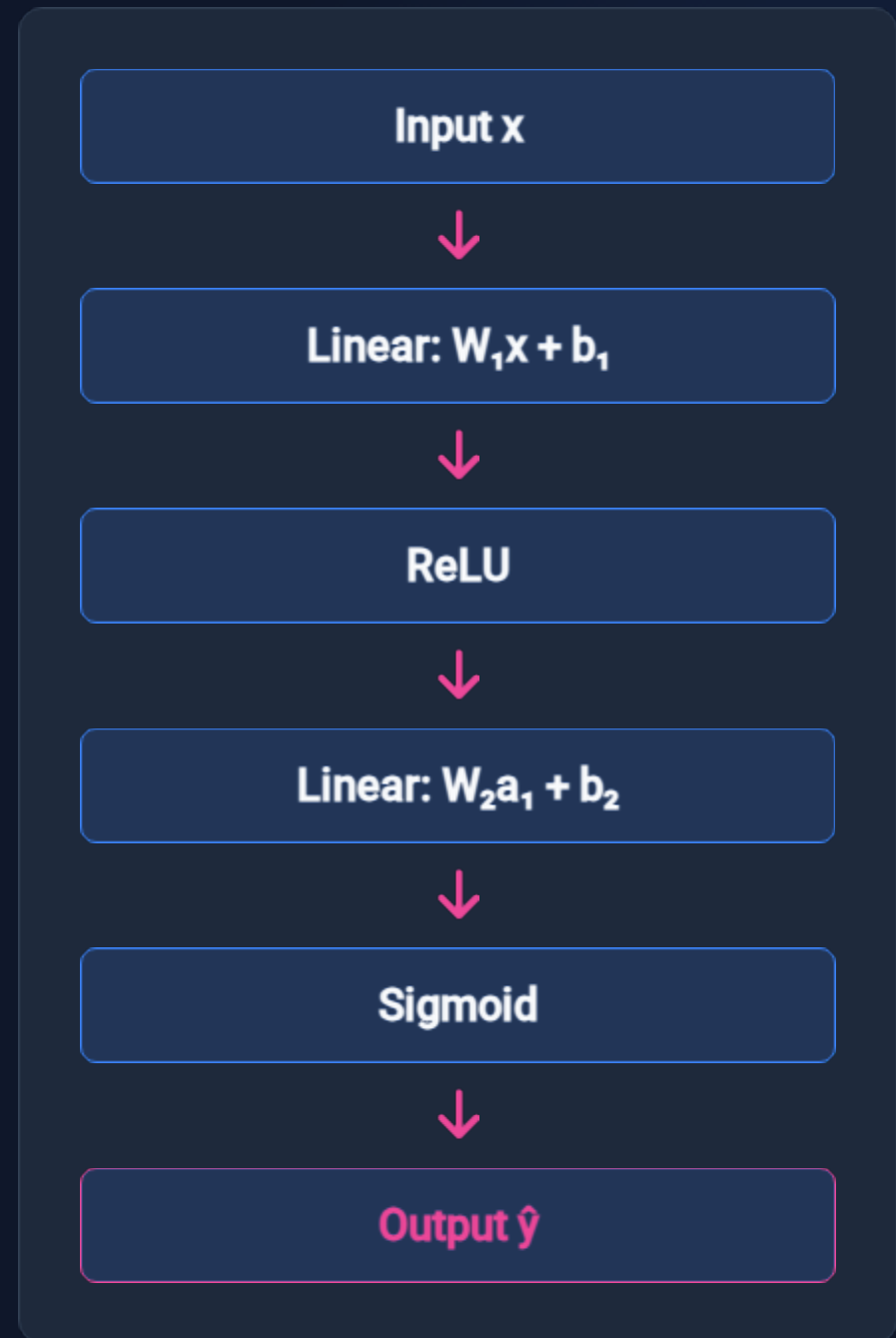
$$z_1 = W_1 x + b_1$$

$$z_2 = W_2 a_1 + b_2$$

$$a_1 = f(z_1)$$

The final output is:

$$\hat{y} = g(z_2)$$



Why Backpropagation?

Suppose the network makes a bad prediction.

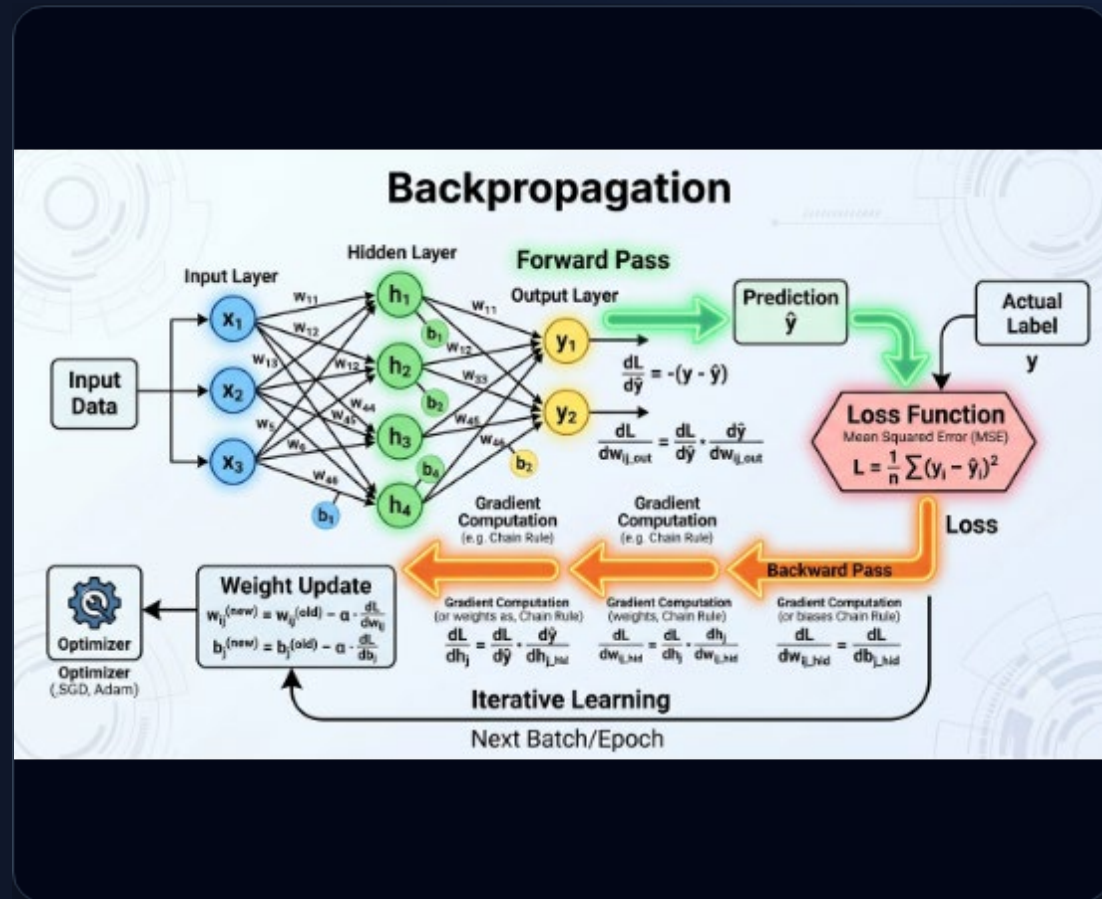
We need to answer:

Which weights caused the error, and by how much should each weight change?

We need the partial derivatives:

$$\frac{\partial L}{\partial W}$$

These derivatives are called Gradients.



Imagine the loss surface as a mountainous landscape.

High Loss

/ \

/ \

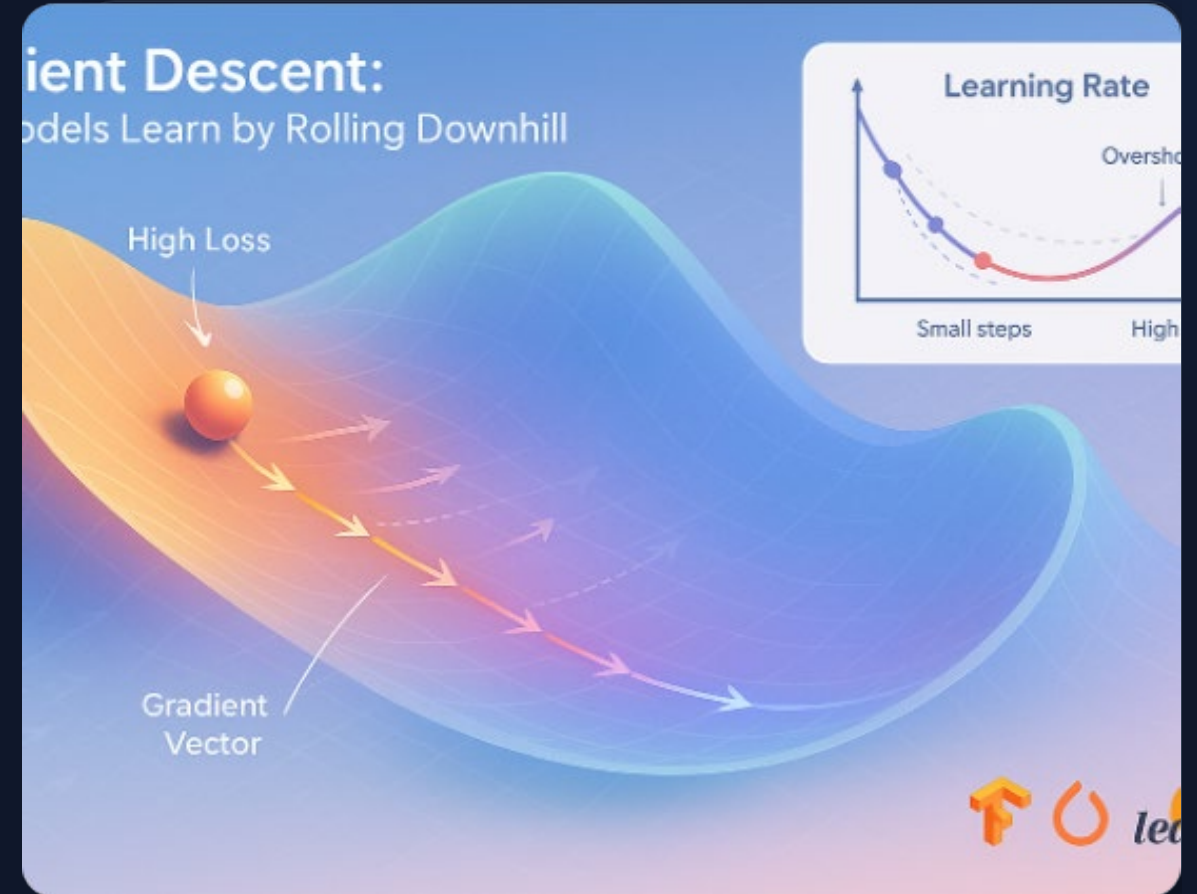
/ \

/ \ _

/ \ _

Minimum

The gradient tells us the direction of steepest increase.



Gradient Descent

The Update Rule

The basic parameter update rule is:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_{\theta} L$$

Variables Explained

- ▶ θ = parameters
- ▶ η = learning rate
- ▶ ∇ = gradient

The learning rate controls the size of the update step.

Learning Rate

TOO SMALL

Tiny updates → Very slow training

APPROPRIATE

Stable progress → Efficient convergence

TOO LARGE

Large jumps → May overshoot → Unstable training





Backpropagation

Efficiently calculating gradients using the Chain Rule.

The Chain Rule

Suppose:

$$L = L(a)$$

and:

$$a = f(z)$$

where:

$$z = wx + b$$

Then, by the Chain Rule:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Since $\frac{\partial z}{\partial w} = x$, we obtain:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot x$$

Backpropagation Direction:



Forward

Input



Hidden



Output



Loss



Backward

Loss



Output



Hidden



Input

Important: Backpropagation does not mean that data is sent backward.

Output-Layer Gradient

For a simple neuron:

$$z = wx + b$$

$$\hat{y} = \sigma(z)$$

And loss:

$$L = L(y, \hat{y})$$

Backpropagation calculates:

$$\frac{\partial L}{\partial w}$$

and:

$$\frac{\partial L}{\partial b}$$

These gradients determine how and should change.

Gradient Through an Activation

Why activation functions matter during training.

Suppose:

$$a = f(z)$$

Then:

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z}$$



Their derivatives directly influence
the gradient flow.

If $\frac{\partial a}{\partial z}$ is zero, the gradient stops flowing
backward.

Backprop Through Layers

For a multi-layer network:

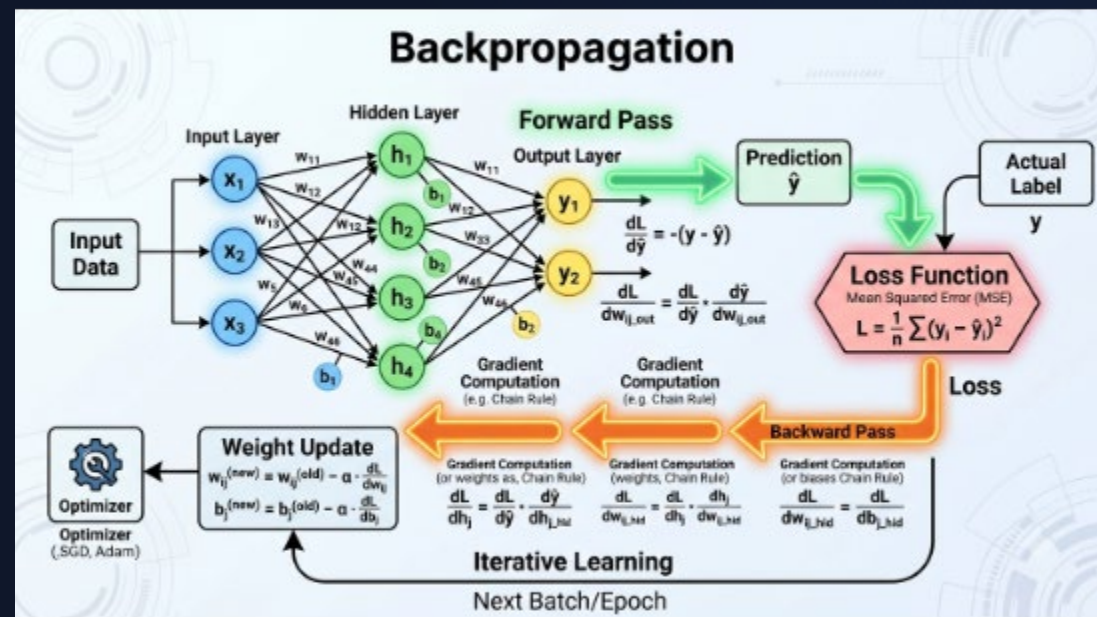
$$x \rightarrow W_1 \rightarrow f \rightarrow W_2 \rightarrow g \rightarrow L$$

The gradient for w_1 depends on the chain of derivatives through later operations.

Conceptually:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial L} \cdots \frac{\partial a_1}{\partial w_1}$$

The chain can contain many factors.



Vanishing Gradients

If many derivatives are small (e.g., Sigmoid derivative is ≤ 0.25):

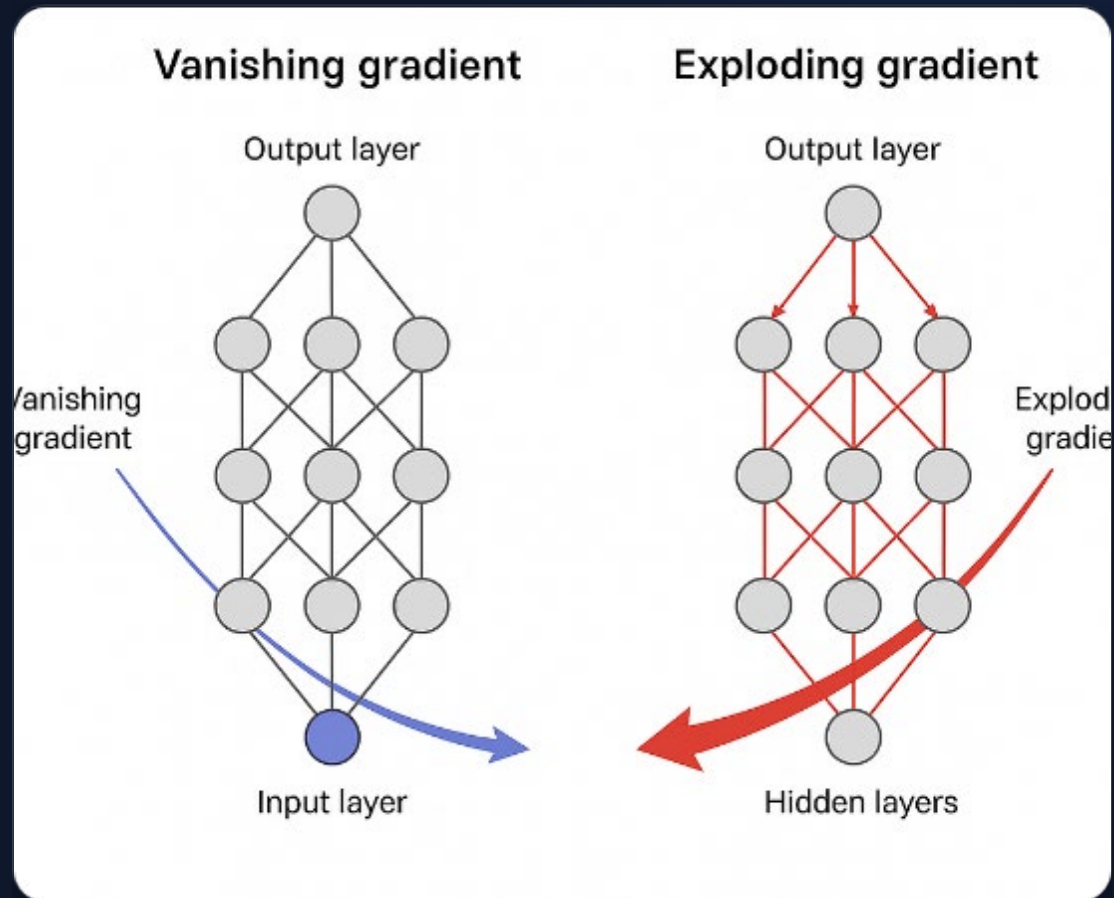
$$0.1 \times 0.1 \times 0.1 \times 0.1 = 0.0001$$

The overall gradient becomes extremely small.

Consequence:

Earlier layers may receive very weak learning signals, meaning their weights barely update.

This is the Vanishing Gradient Problem.



Exploding Gradients

The opposite can also happen.

If gradient factors are large:

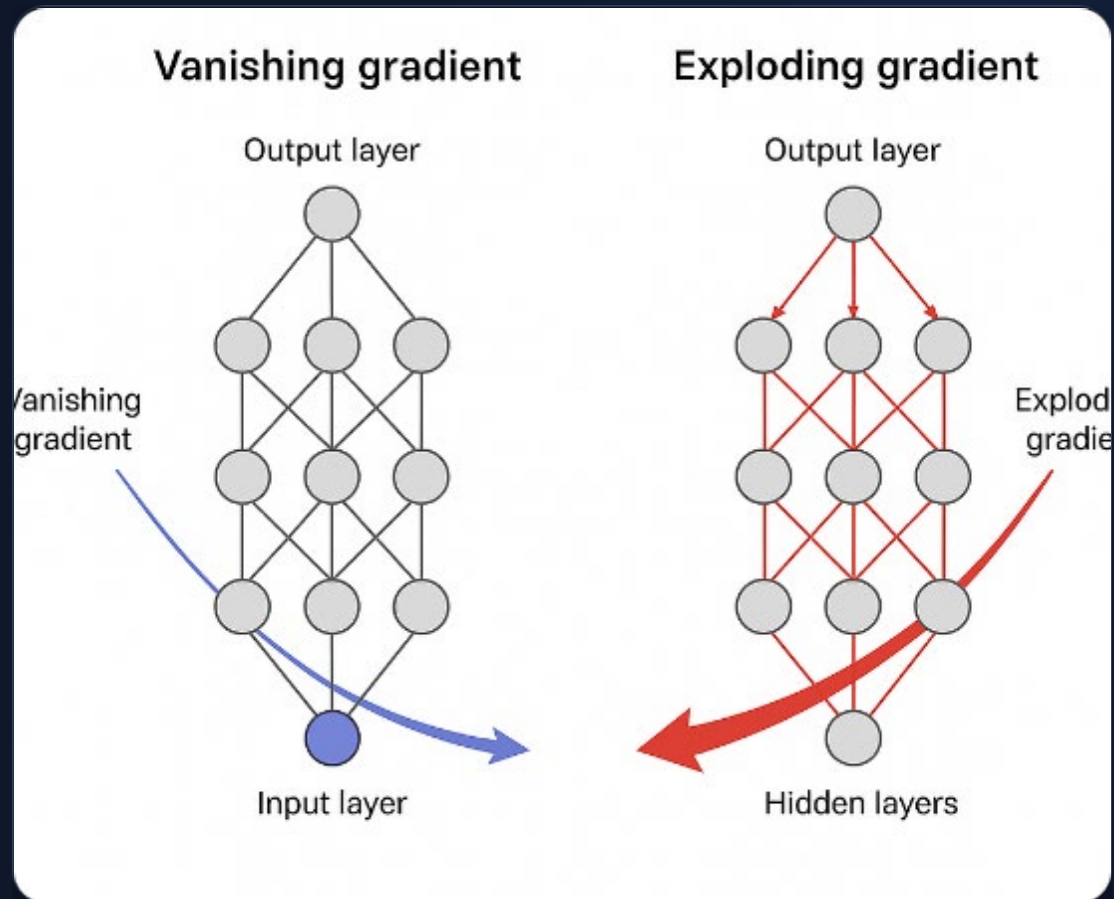
$$2 \times 2 \times 2 \times 2 \times 2 = 32$$

Gradients can grow rapidly.

Consequences

- Very large updates
- Unstable training
- Numerical problems (NaNs)

This is the Exploding Gradient Problem.



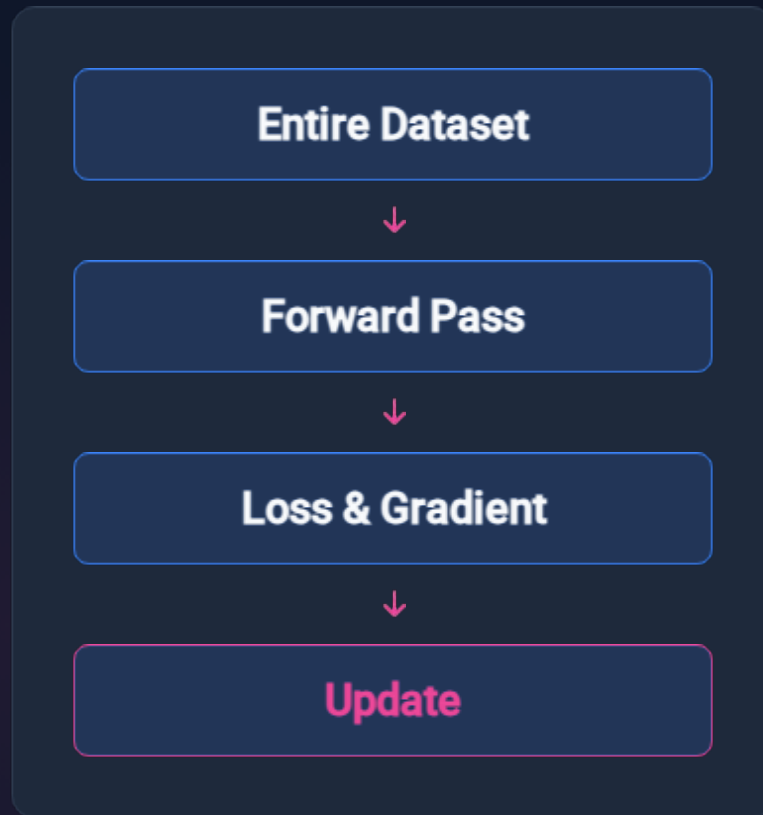


Optimization Techniques

Methods to apply gradients efficiently and effectively.

Batch Gradient Descent

Compute the gradient using the entire training dataset.



Advantage

Stable gradient estimate. Provides a true direction of the loss surface.

Disadvantage

Can be computationally expensive and painfully slow for large datasets. Doesn't fit in memory.

SGD updates parameters using one training example at a time.

Example 1 → Update

Example 2 →
Update

Example 3 →
Update

...

Advantages

- ▶ Frequent updates
- ▶ Can escape some poor local minima
- ▶ Efficient for large datasets

Disadvantage

Updates can be very noisy and erratic.

Mini-Batch Gradient Descent

Instead of the entire dataset or just one example, use a **small batch**.

Dataset Split into Batches



Batch 1 → Update

Batch 2 → Update

Batch 3 → Update



Mini-batch training is extremely common in deep learning.

It balances stability and speed.

Batch Size

16, 32, 64...

Typical Examples

Choosing the Batch Size

Batch size is a hyperparameter.

A larger batch:

- ▶ Provides a smoother gradient estimate
- ▶ May require more memory (VRAM)

A smaller batch:

- ▶ Uses less memory
- ▶ Produces noisier updates

Momentum

Momentum incorporates information from previous updates.

Conceptually:

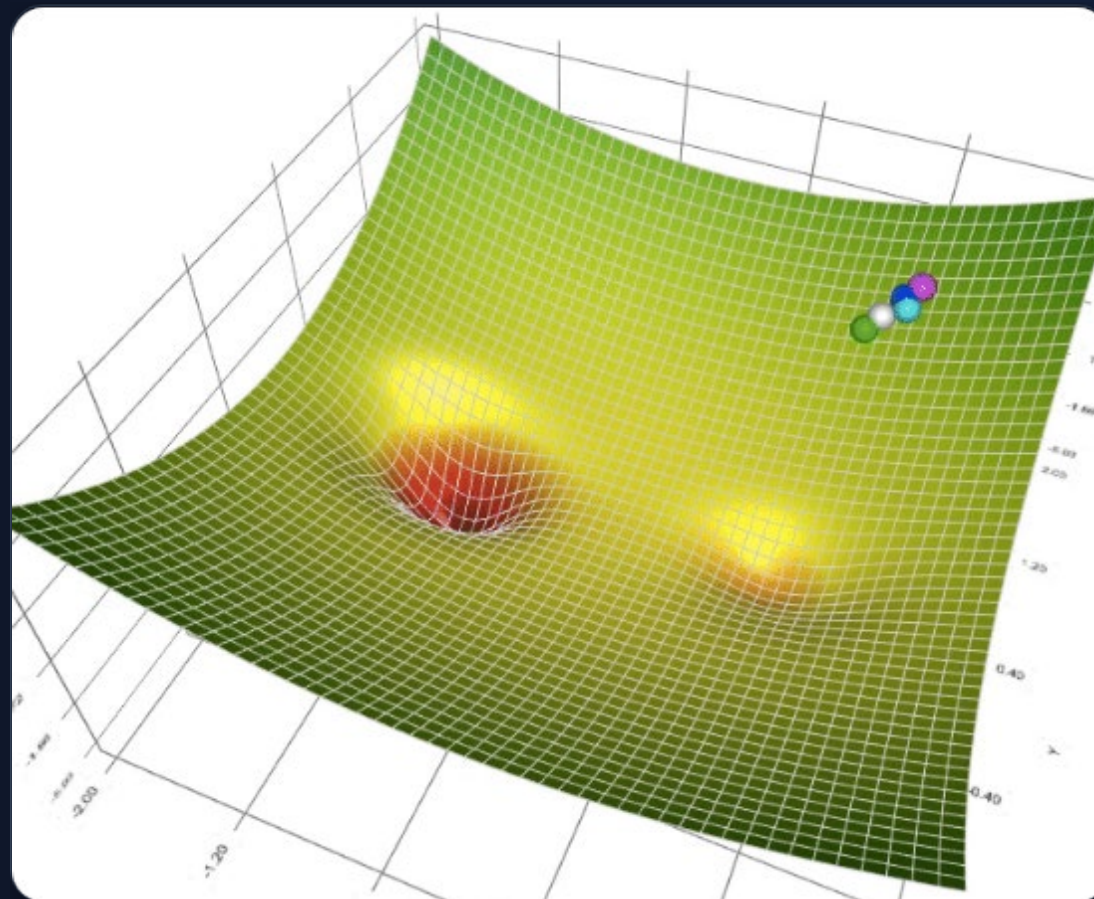
$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + (1 - \beta) \mathbf{g}_t$$

and:

$$\theta_t = \theta_{t-1} - \eta \mathbf{v}_t$$

Intuition:

Instead of reacting only to the current gradient, the optimizer maintains a form of velocity. This helps accelerate learning in consistent directions.



Adam Optimizer

Adaptive Moment Estimation.

Adam combines ideas related to:

- ▶ Momentum
- ▶ Adaptive learning rates

It maintains moving estimates of:

- ▶ First moment of gradients (mean)
- ▶ Second moment of gradients (variance)

Adam is widely used because it often provides a strong practical starting point.

⚠ But: Adam is not automatically the best optimizer for every task.

Learning Rate Scheduling

A fixed learning rate is not always ideal.
We can change it during training.

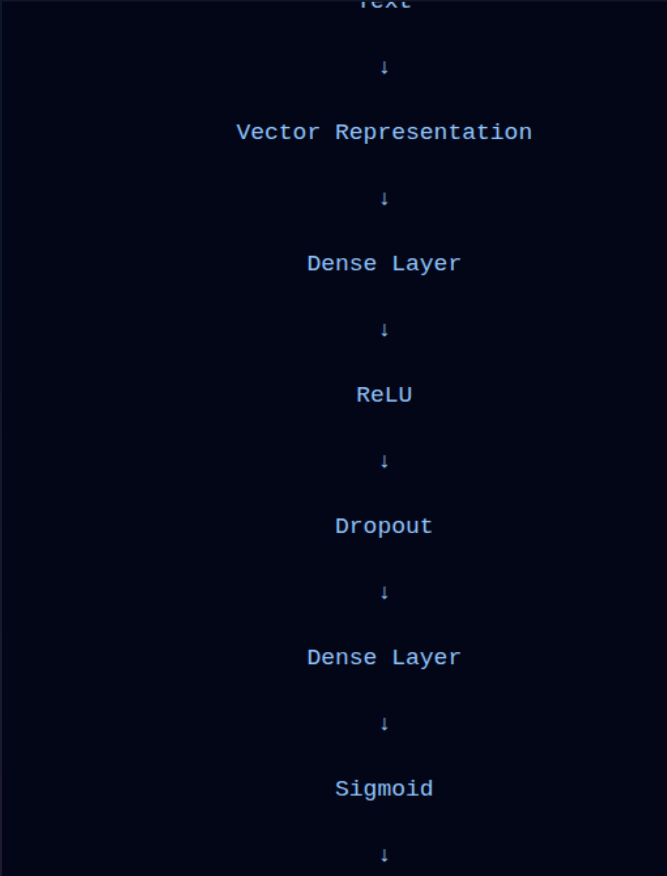
Possible Strategies:

- ▶ Step decay
- ▶ Exponential decay
- ▶ Cosine schedules
- ▶ Warm-up followed by decay

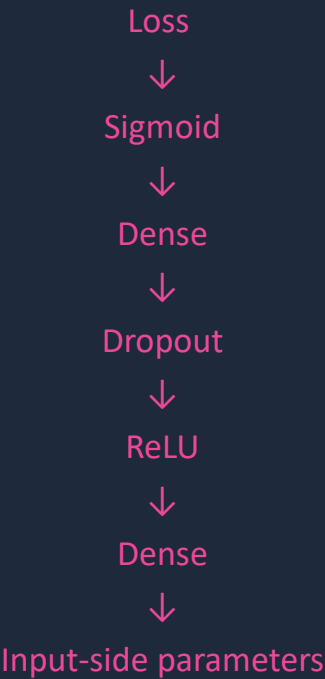


Backprop in an NLP Classifier

Forward Pass



During Backpropagation



Gradients update the trainable weights.

Summary & Next Lecture



Key Takeaways

- ▶ **Backpropagation:** Efficiently computes gradients using the chain rule.
- ▶ **Gradient descent:** Updates parameters in the direction that reduces loss (θ).
$$\theta \leftarrow \theta - \eta \nabla L$$
- ▶ **Optimization:** Batch, SGD, Mini-batch, Momentum, Adam, LR schedules.
- ▶ **Key problems:** Vanishing and exploding gradients.



Next Lecture

Convolutional Networks in Text Classification

We move from:

Dense networks $\rightarrow$ learning with gradients

to:

How can neural networks detect local patterns such as phrases and word sequences in text?

Image Sources



<https://www.programming-partner.com/uploads/backpropagation-explained-neural-networks-learn-image-0.jpeg>

Source: www.programming-partner.com



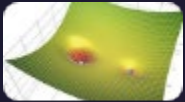
https://miro.medium.com/v2/resize:fit:1400/1*Qp5GTUYrRMue4EnFONAayw.png

Source: [medium.com](https://miro.medium.com)



https://miro.medium.com/v2/resize:fit:1200/1*z7Dpo68uOaKjxbzxVYJ3g.png

Source: [medium.com](https://miro.medium.com)



<https://assets.insightmediagroup.io/media/wp-content/uploads/2025/03/image-18.gif>

Source: [towardsdatascience.com](https://assets.insightmediagroup.io)